

MATH00041: Introduction to Programming

Text Strings - Numbers (Booleans, Integers, Floats, Complex)

Lecture 2

Archontis Giannakidis, PhD

Outline

- **Text strings**
- **Booleans**
- **Integers**
- **Floats**
- **Complex**

Strings

In Python, when you create a string, you can enclose it in either single quotes or double quotes. For example, the strings `'hello'` and `"hello"` are equivalent. Both will produce the same output when printed. This allows you to pick your preferred type of quotation marks.

You can display a string literal with the `print()` function:

Example:

```
print("Hello")  
print('Hello')
```

output: Hello

output: Hello

Assign String to a Variable

Assigning a string to a variable is done with the variable name followed by an equal sign and the string:

Example:

```
a = "Hello"  
print(a)
```

output: Hello

Multiline Strings

You can assign a multiline string to a variable by using three quotes:

Example:

```
a = """The Modern University of the Year for 2023 was awarded  
to NTU primarily based on its continuous investment in teaching  
quality, student experience and graduate prospects."""
```

```
print(a)
```

output: The Modern University of the Year for 2023 was awarded to NTU primarily based on its continuous investment in teaching quality, student experience and graduate prospects.

Strings are Arrays.

Like many other popular programming languages, strings in Python are arrays of bytes representing Unicode characters.

However, Python does not have a character data type; a single character is simply a string with a length of 1.

Square brackets can be used to access elements of the string.

Example:

Get the character at position 1 (remember that the first character has the position 0):

```
a = "Hello, World!"  
print(a[1])
```

output: World!

Looping Through a String

Since strings are arrays, we can loop through the characters in a string, with a `for` loop.

Example:

Loop through the letters in the word "banana":

```
for x in "banana":  
    print(x)
```

output: b

output: a

output: n

output: a

output: n

output: a

String Length

To get the length of a string, use the `len()` function.

Example:

The `len()` function returns the length of a string:

```
a = "Hello, World!"  
print(len(a))
```

output: 13

Check String

To check if a certain phrase or character is present in a string, we can use the keyword `in`.

Example:

Check if "free" is present in the following text:

```
txt = "The best things in life are free!"  
print("free" in txt)
```

output: True

Use it in an `if` statement:

Example:

Print only if "free" is present:

```
txt = "The best things in life are free!"  
if "free" in txt:  
    print("Yes, 'free' is present.")
```

output: Yes, 'free' is present.

Check if NOT

To check if a certain phrase or character is NOT present in a string, we can use the keyword `not in`.

Example:

Check if "expensive" is NOT present in the following text:

```
txt = "The best things in life are free!"  
print("expensive" not in txt)
```

output: True

Use it in an `if` statement:

Example:

print only if "expensive" is NOT present:

```
txt = "The best things in life are free!"  
if "expensive" not in txt:  
    print("No, 'expensive' is NOT present.")
```

output: No, 'expensive' is NOT present.

Slicing

You can return a range of characters by using the slice syntax.

Specify the start index and the end index, separated by a colon, to return a part of the string.

Example:

Get the characters from position 2 to position 5 (not included):

```
b = "Hello, World!"
```

```
print(b[2:5])
```

output: llo

Slice From the Start

By leaving out the start index, the range will start at the first character:

Example:

Get the characters from the start to position 5 (not included):

```
b = "Hello, World!"  
print(b[:5])
```

output: Hello

Slice To the End

By leaving out the end index, the range will go to the end:

Example:

Get the characters from position 2, and all the way to the end:

```
b = "Hello, World!"  
print(b[2:])
```

output: llo, World!

Negative Indexing

Use negative indexes to start the slice from the end of the string:

Example:

Get the characters:

From: "o" in "World!" (position -5)

To, but not included: "d" in "World!" (position -2):

```
b = "Hello, World!"
```

```
print(b[-5:-2])
```

output: orl

Upper Case

Example:

The upper() method returns the string in upper case:

```
a = "Hello, World!"  
print(a.upper())
```

output: HELLO, WORLD!

Lower Case

Example:

The lower() method returns the string in lower case:

```
a = "Hello, World!"  
print(a.lower())
```

output: hello, world!

Remove Whitespace

Whitespace is the space before and/or after the actual text, and very often you want to remove this space.

Example:

The strip() method removes any whitespace from the beginning or the end:

```
a = " Hello, World! "
```

```
print(a.strip()) # returns "Hello, World!"
```

output: Hello, World!

Replace String

Example:

The replace() method replaces a string with another string:

```
a = "Hello, World!"  
print(a.replace("H", "J"))
```

output: Jello, World!

Split String

The `split()` method returns a list where the text between the specified separator becomes the list items.

Example:

The `split()` method splits the string into substrings if it finds instances of the separator:

```
a = "Hello, World!"  
print(a.split(",")) # returns ['Hello', ' World!']
```

output: ['Hello', ' World!']

String Concatenation

To concatenate, or combine, two strings you can use the + operator.

Example:

Merge variable a with variable b into variable c:

```
a = "Hello"
```

```
b = "World"
```

```
c = a + b
```

```
print(c)
```

output: HelloWorld

Example:

To add a space between them, add a " ":

```
a = "Hello"
```

```
b = "World"
```

```
c = a + " " + b
```

```
print(c)
```

output: Hello World

String Format

As we learned in the Python Variables chapter, we cannot combine strings and numbers like this:

Example:

Adding a string and Integer is not permitted.

```
age = 36
txt = "My name is John, I am " + age
print(txt)
```

output:

Traceback (most recent call last):

File "demo_string_format_error.py", line 2, in <module>

txt = "My name is John, I am " + age

TypeError: must be str, not int

But we can combine strings and numbers by using the `format()` method!

The `format()` method takes the passed arguments, formats them, and places them in the string where the placeholders `{}` are:

Example:

Use the `format()` method to insert numbers into strings:

```
age = 36  
txt = "My name is John, and I am {}"  
print(txt.format(age))
```

output: My name is John, and I am 36

The `format()` method takes unlimited number of arguments, and are placed into the respective placeholders:

Example:

```
quantity = 3  
itemno = 567  
price = 49.95  
myorder = "I want {} pieces of item {} for {} dollars."  
print(myorder.format(quantity, itemno, price))
```

output: I want 3 pieces of item 567 for 49.95 dollars.

You can use index numbers {0} to be sure the arguments are placed in the correct placeholders:

Example:

```
quantity = 3
```

```
itemno = 567
```

```
price = 49.95
```

```
myorder = "I want to pay {2} dollars for {0} pieces of item  
{1}."
```

```
print(myorder.format(quantity, itemno, price))
```

output: I want to pay 49.95 dollars for 3 pieces of item 567

Escape Character

To insert characters that are illegal in a string, use an escape character.

An escape character is a backslash \ followed by the character you want to insert.

An example of an illegal character is a double quote inside a string that is surrounded by double quotes:

Example:

You will get an error if you use double quotes inside a string that is surrounded by double quotes:

```
txt = "We are the so-called "Vikings" from the north."
```

output:

File "demo_string_escape_error.py", line 1

```
txt = "We are the so-called "Vikings" from the north."
```

^

SyntaxError: invalid syntax

To fix this problem, use the escape character `\`:

Example:

The escape character allows you to use double quotes when you normally would not be allowed:

```
txt = "We are the so-called \"Vikings\" from the north."
```

output: We are the so-called "Vikings" from the north.

Escape Characters

Other escape characters used in Python

Symbol	Result	Example	Output
'	Single Quote	<code>print('It\'s alright.')</code>	It's alright.
\\	Backslash	<code>print("This will insert one \\ (backslash).")</code>	This will insert one \ (backslash).
\n	New Line	<code>print("Hello\nWorld!")</code>	Hello World!
\r	Carriage Return	<code>print("Hello\rWorld!")</code>	Hello World!
\t	Tab	<code>print("Hello\tWorld!")</code>	Hello World!
\b	Backspace	<code>print("Hello \bWorld!")</code>	HelloWorld!
\f	Form Feed	<code>with open('hello_world.txt', 'w') as f: f.write("Hello World\f")</code>	
\ooo	Octal value	<code>print("\110\145\154\154\157")</code>	Hello
\xhh	Hex value	<code>print("\x48\x65\x6c\x6c\x66")</code>	Hello

String Methods

Python has a set of built-in methods that you can use on strings.

Method	Description
capitalize()	Converts the first character to upper case
casefold()	Converts string into lower case
center()	Returns a centered string
count()	Returns the number of times a specified value occurs in a string
encode()	Returns an encoded version of the string
endswith()	Returns true if the string ends with the specified value
expandtabs()	Sets the tab size of the string
find()	Searches the string for a specified value and returns the position of where it was found
format()	Formats specified values in a string
format_map()	Formats specified values in a string
index()	Searches the string for a specified value and returns the position of where it was found

isalnum()	Returns True if all characters in the string are alphanumeric
isalpha()	Returns True if all characters in the string are in the alphabet
isascii()	Returns True if all characters in the string are ascii characters
isdecimal()	Returns True if all characters in the string are decimals
isdigit()	Returns True if all characters in the string are digits
isidentifier()	Returns True if the string is an identifier
islower()	Returns True if all characters in the string are lower case
isnumeric()	Returns True if all characters in the string are numeric
isprintable()	Returns True if all characters in the string are printable

<code>isspace()</code>	Returns True if all characters in the string are whitespaces
<code>istitle()</code>	Returns True if the string follows the rules of a title
<code>isupper()</code>	Returns True if all characters in the string are upper case
<code>join()</code>	Joins the elements of an iterable to the end of the string
<code>ljust()</code>	Returns a left justified version of the string
<code>lower()</code>	Converts a string into lower case
<code>lstrip()</code>	Returns a left trim version of the string
<code>maketrans()</code>	Returns a translation table to be used in translations
<code>partition()</code>	Returns a tuple where the string is parted into three parts
<code>replace()</code>	Returns a string where a specified value is replaced with a specified value

<code>rfind()</code>	Searches the string for a specified value and returns the last position of where it was found
<code>rindex()</code>	Searches the string for a specified value and returns the last position of where it was found
<code>rjust()</code>	Returns a right justified version of the string
<code>rpartition()</code>	Returns a tuple where the string is parted into three parts
<code>rsplit()</code>	Splits the string at the specified separator, and returns a list
<code>rstrip()</code>	Returns a right trim version of the string
<code>split()</code>	Splits the string at the specified separator, and returns a list
<code>splitlines()</code>	Splits the string at line breaks and returns a list
<code>startswith()</code>	Returns true if the string starts with the specified value
<code>strip()</code>	Returns a trimmed version of the string
<code>swapcase()</code>	Swaps cases, lower case becomes upper case and vice versa
<code>title()</code>	Converts the first character of each word to upper case
<code>translate()</code>	Returns a translated string
<code>upper()</code>	Converts a string into upper case
<code>zfill()</code>	Fills the string with a specified number of 0 values at the beginning

Booleans

Booleans represent one of two values: **True** or **False**.

In programming, you often need to know if an expression is **True** or **False**.

You can evaluate any expression in Python and get one of two answers: **True** or **False**.

When you compare two values, the expression is evaluated, and Python returns the Boolean answer:

Example:

```
print(10 > 9)
print(10 == 9)
print(10 < 9)
```

output: True

output: False

output: False

When you run a condition in an if statement, Python returns **True** or **False**:

Example:

Print a message based on whether the condition is True or False:

```
a = 200
```

```
b = 33
```

```
if b > a:
    print("b is greater than a")
else:
    print("b is not greater than a")
```

output: b is not greater than a

Evaluate Values and Variables

The `bool()` function allows you to evaluate any value and give you **True** or **False** in return,

Example:

Evaluate a string and a number:

```
print(bool("Hello"))
```

```
print(bool(15))
```

output: True

output: True

Example:

Evaluate two variables:

```
x = "Hello"
```

```
y = 15
```

```
print(bool(x))
```

```
print(bool(y))
```

output: True

output: True

Most Values are True

Almost any value is evaluated to **True** if it has some sort of content.

Any string is **True**, except empty strings.

Any number is **True**, except 0.

Any list, tuple, set, and dictionary are **True**, except empty ones.

Example:

The following will return True:

```
bool("abc")
```

```
bool(123)
```

```
bool(["apple", "cherry", "banana"])
```

output: True

output: True

output: True

Some Values are False

In fact, there are not many values that evaluate to **False**, except empty values, such as `()`, `[]`, `{}`, `""`, the number `0`, and the value `None`. And of course, the value **False** evaluates as **False**.

Example:

The following will return False:

```
bool(False), bool(None), bool(0), bool(""), bool(()), bool([]), bool({})
```

output: False, False, False, False, False, False, False

One more value, or object in this case, evaluates to **False**, and that is if you have an object that is made from a class with a **__len__** function that returns **0** or **False**:

Example:

```
class myclass():  
    def __len__(self):  
        return 0  
myobj = myclass()  
print(bool(myobj))
```

output: False

Functions can Return a Boolean

You can create functions that returns a Boolean Value:

Example:

Print the answer of a function:

```
def myFunction() :  
    return True  
print(myFunction())
```

output: True

You can execute code based on the Boolean answer of a function:

Example:

Print "YES!" if the function returns True, otherwise print "NO!":

```
def myFunction() :  
    return True  
if myFunction():  
    print("YES!")  
else:  
    print("NO!")
```

output: YES!

Python also has many built-in functions that return a boolean value, like the `isinstance()` function, which can be used to determine if an object is of a certain data type:

Example:

Check if an object is an integer or not:

```
x = 200  
print(isinstance(x, int))
```

output: True

Numeric Data Type in Python

Python's numeric data type is used to represent data with a numeric value. Numeric values in Python can be integers, floating point numbers or complex numbers. These values are defined as classes in Python - int, float and complex.

- Integers – This value is represented by the int class in Python. It holds whole numbers, both positive and negative, without any decimals or fractions. Python's int class has no limit on the size of integer values.
- Float – A floating-point number is represented by the float class. It is a real number that has a decimal point in its representation. To denote scientific notation, you can optionally append the character 'e' or 'E' followed by a positive or negative integer.
- Complex Numbers – A complex class represents a complex number. It is specified as (real part) + (imaginary part)j. For example – 2+3j
- Note – type() function determines the data type.

Example:

demonstrate numeric value:

```
x = 200
```

```
print(isinstance(x, int))
```

```
a = 5
```

```
print("Type of a: ", type(a))
```

```
b = 5.0
```

```
print("\nType of b: ", type(b))
```

```
c = 2 + 4j
```

```
print("\nType of c: ", type(c))
```

```
output: Type of a: <class 'int'>
```

```
output: Type of b: <class 'float'>
```

```
output: Type of c: <class 'complex'>
```

int() Function

Syntax: `int(x, base)`

- `x` [optional]: string representation of integer value, defaults to 0 if no value is provided.
- `base` [optional]: (integer value) base of the number.

Returns: Return decimal (base-10) representation of `x`

Example:

examples of `int()` in Python:

```
# int() on string representation of numbers
print("int('9') =", int('9'))
```

```
# int() on float values
print("int(9.9) =", int(9.9))
```

```
# int() on Python integer
print("int(9) =", int(9))
```

output : `int('9') = 9`

output : `int(9.9) = 9`

output : `int(9) = 9`

Convert base using int() in Python

In the example below, we utilized the int() function to convert a number's base from Binary, Octal, and Hexadecimal to a Decimal integer.

Example:

examples of int() in Python:

```
# octal to decimal using int()
print("int() on 0o12 =", int('0o12', 8))

# binary to decimal using int()
print("int() on 0b110 =", int('0b110', 2))

# hexa-decimal to decimal using int()
print("int() on 0x1A =", int('0x1A', 16))

output : int() on 0o12 = 10
output : int() on 0b110 = 6
output : int() on 0x1A = 26
```

float() function

Syntax: float(x)

Parameter x: x is optional & can be:

- any number or number in form of string, ex,: “10.5”
- inf or infinity, NaN (any cases)

Return: Float Value

Values that the Python float() method can return depending upon the argument passed.

- If an argument is passed, then the equivalent floating-point number is returned.
- If no argument is passed, then the method returns 0.0.
- If any string is passed that is not a decimal point number or does not match any cases mentioned above, then an error will be raised.
- If a number is passed outside the range of Python float, then OverflowError is generated.

Python `float()` function is used to return a floating-point number from a number or a string representation of a numeric value.

Example:

Python `float()` function which takes an integer as the parameter and returns its float value:

```
num = float(10)
print(num)
```

output : 10.0

Let us see how the Python `float()` function works, In this example, we passed different datatype values to the `float()` function as parameters to see how it works.

Example:

Various examples and working of `float()`:

```
print(float(21.89))
```

```
# for floating point numbers
```

```
print (float (8))
```

```
# for integer type strings
```

```
print (float ("23"))
```

```
# for floating type strings
```

```
print (float ("-16.54"))
```

```
# for string floats with whitespaces
```

```
print (float ("    -24.45    \n"))
```

```
# for inf/infinity
```

```
print (float ("InF"))
```

```
print (float ("InFiNiTy"))
```

```
# for NaN
```

```
print (float ("nan"))
```

```
print (float ("NaN"))
```

output : 21.89

output : 8.0

output : 23.0

output : -16.54

output : -24.45

output : inf

output : inf

output : nan

output : nan

complex() Function

Python `complex()` function returns a complex number (real + imaginary) example (5+2j) when real and imaginary parts are passed, or it also converts a string to a complex number.

Syntax: `complex ([real[, imaginary]])`

- `real [optional]`: numeric type (including complex). It defaults to zero.
- `imaginary [optional]`: numeric type (including complex) .It defaults to zero.

Return: Returns a complex number in the form of (real + imaginary) example (5+2j)

- ❖ **Note:** If the first parameter passed is a string, the second parameter should not be passed, otherwise a `TypeError` will be raised. The string must not contain whitespace around the + or – operator, otherwise a `ValueError` will be raised in Python.

Example:

```
# nothing is passed
```

```
z = complex()
```

```
print("complex() with no parameters:", z)
```

```
# integer type
```

```
# passing first parameter only
```

```
complex_num1 = complex(5)
```

```
print("Int: first parameter only", complex_num1)
```

```
# passing both parameters
```

```
complex_num2 = complex(7, 2)
```

```
print("Int: both parameters", complex_num2)
```

```
# float type
```

```
# passing first parameter only
```

```
complex_num3 = complex(3.6)
```

```
print("Float: first parameter only", complex_num3)
```

```
# passing both parameters
```

```
complex_num4 = complex(3.6, 8.1)
```

```
print("Float: both parameters", complex_num4)
```

```
# type
```

```
print(type(complex_num1))
```

output : complex() with no parameters: 0j

output : Int: first parameter only (5+0j)

output : Int: both parameters (7+2j)

output : Float: first parameter only (3.6+0j)

output : Float: both parameters (3.6+8.1j)

output : <class 'complex'> output : nan

Thank you